ADVENTIST UNIVERSITY OF CENTRAL AFRICA

MSDA9215 ADVANCED BIG DATA ANALYTICS · ASSIGNMENT I

Smart energy grid analytics

An MQTT and TimescaleDB pipeline for 4,000 smart meters, from a pilot stream to a next-day forecast

GROUP 10

Thierry SEBAKUNZI	101394
Kwezi James	101502
Nishyirimbere Yves	101388

Instructor: Temitope Oguntade

OCTOBER 2026

10,696,848	2.72×	725×	5.8×
readings loaded	smaller storage	faster dashboard	lower forecast error

Synthetic smart-meter readings from the instructor-provided generator, `smart_meter_simulator.py`, used unchanged.

CONTENTS

Contents

1 **Build the ingestion pipeline**
1.1 Confirm the environment · 1.2 Create the database structures · 1.3 Capture and publish a pilot stream

2 **Capture and prepare the historical dataset**
2.1 Capture and load the complete stream · 2.2 Investigate data quality · 2.3 Clean and preserve a consistent benchmark

3 **Compare chunk intervals**
3.1 Create and verify three configurations · 3.2 Implement and time the five workloads · 3.3 Explain the chunk results

4 **Compare compression strategies**
4.1 Create the two compressed configurations · 4.2 Measure storage and compression cost · 4.3 Repeat the query comparison · 4.4 Select a storage configuration

5 **Investigate the utility data**
5.1 Identify demand patterns · 5.2 Compare regions and customer types · 5.3 Assess reporting quality · 5.4 Investigate abnormal consumption · 5.5 Management recommendation

6 **Create and investigate continuous aggregates**
6.1 Create the three aggregate views · 6.2 Test freshness and delayed arrivals

7 **Build and compare dashboards**
7.1 and 7.2 Dashboards · 7.3 Verify matching results · 7.4 Measure performance and judge the tradeoff

8 **Forecast the next day**
8.1 Prepare chronological data and features · 8.2 Previous-day baseline · 8.3 Train and evaluate one regression model · 8.4 Judge forecast usefulness

9 **Submit the report and code**
9.1 Organise the report · 9.2 Make the work reproducible · 9.3 Document contributions

Management recommendation
Section 5.5, at most 500 words

GitHub repository with full code, SQL, logs and screenshots: `github.com/code-gnosis/Group10_Big_Data_Analytics_assignment_1`

All screenshots show the macOS menu bar with the clock and the Dock, as the brief asks.

1 Build the ingestion pipeline

1.1 Confirm the environment 3 MARKS

Table 1. Host, software and PostgreSQL settings.

ITEM	VALUE
CPU	`Apple M4`
Cores	`10 physical / 10 logical`
RAM	`16 GB`
OS	`macOS 26.6.2 (build 25G83)`
Storage	`Apple Fabric (NVMe)`
Python	`3.14.7`
EMQX	`5.7.2 (Docker container)`
PostgreSQL	`16.15`
TimescaleDB	`2.30.2`
Deployment	`Docker container (no CPU/RAM limits set)`
`shared_buffers`	`253824 (2 GB)`
`work_mem`	`6347 kB`
`max_parallel_workers_per_gather`	`5`
`jit`	`off`

Database `energy_grid` was created with `TimeZone = Africa/Kigali` so daily buckets align with CAT midnight.



Screenshot 1. Environment report from `env_report.sh`: hardware, OS, software versions, PostgreSQL settings, database time zone and EMQX broker status.

1.2 Create the database structures 3 MARKS

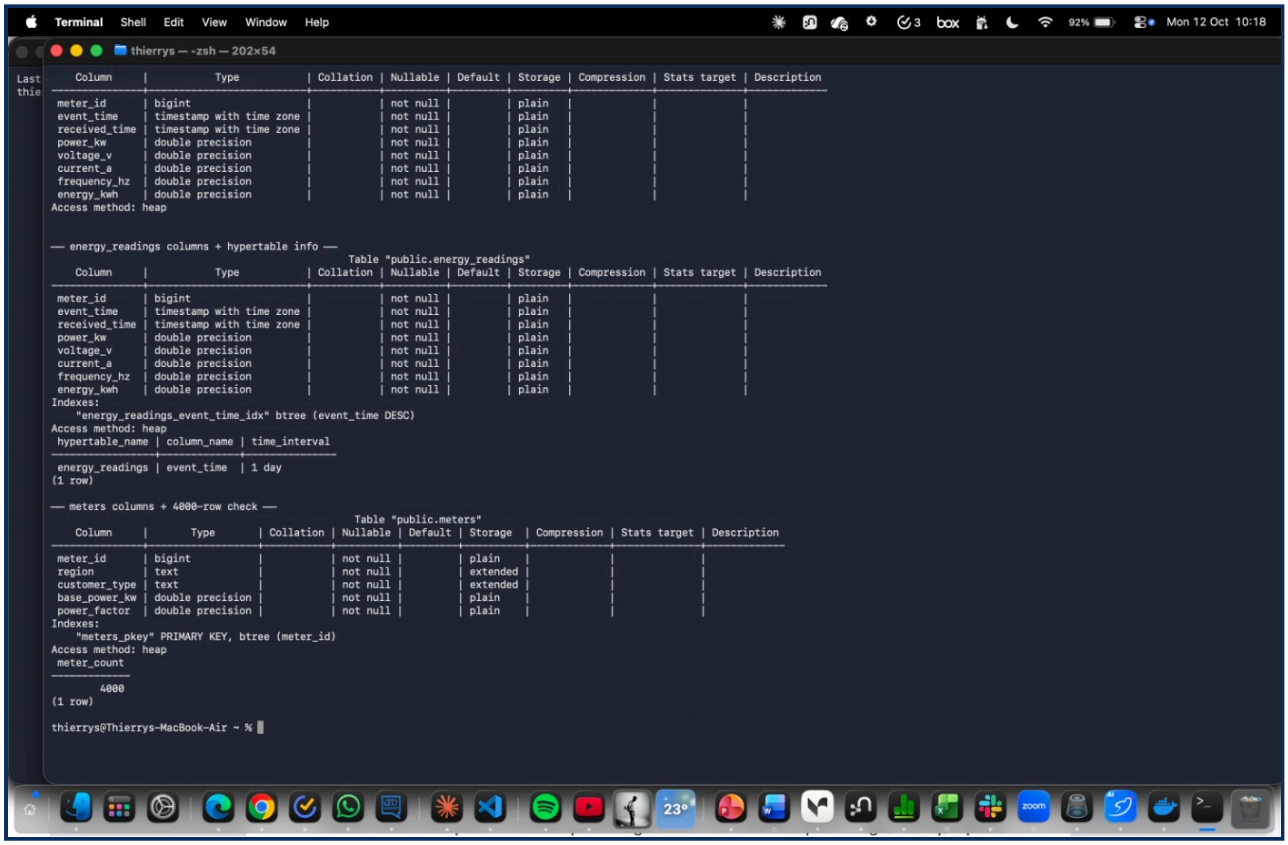
Three tables are created in `sql/02_schema.sql`.

- `energy_live`: a regular table for the pilot.
- `energy_readings`: a TimescaleDB **hypertable** with 1-day chunks. A hypertable is a table that TimescaleDB splits automatically into time slices called **chunks**.
- `meters`: meter metadata, with `meter_id` as the primary key.

Table 2. The 8 columns shared by both reading tables.

COLUMN	TYPE
`meter_id`	`BIGINT`
`event_time`	`TIMESTAMPTZ`
`received_time`	`TIMESTAMPTZ`
`power_kw`	`DOUBLE PRECISION`
`voltage_v`	`DOUBLE PRECISION`
`current_a`	`DOUBLE PRECISION`
`frequency_hz`	`DOUBLE PRECISION`
`energy_kwh`	`DOUBLE PRECISION`

The loader script `code/02_load_meters.py` calls `meter_metadata()` and inserts the 4,000 rows into `meters`. The count check returns **4000**.



Screenshot 2. Column definitions for both reading tables, the hypertable's 1-day chunk interval, and the 4,000-row check on `meters`.

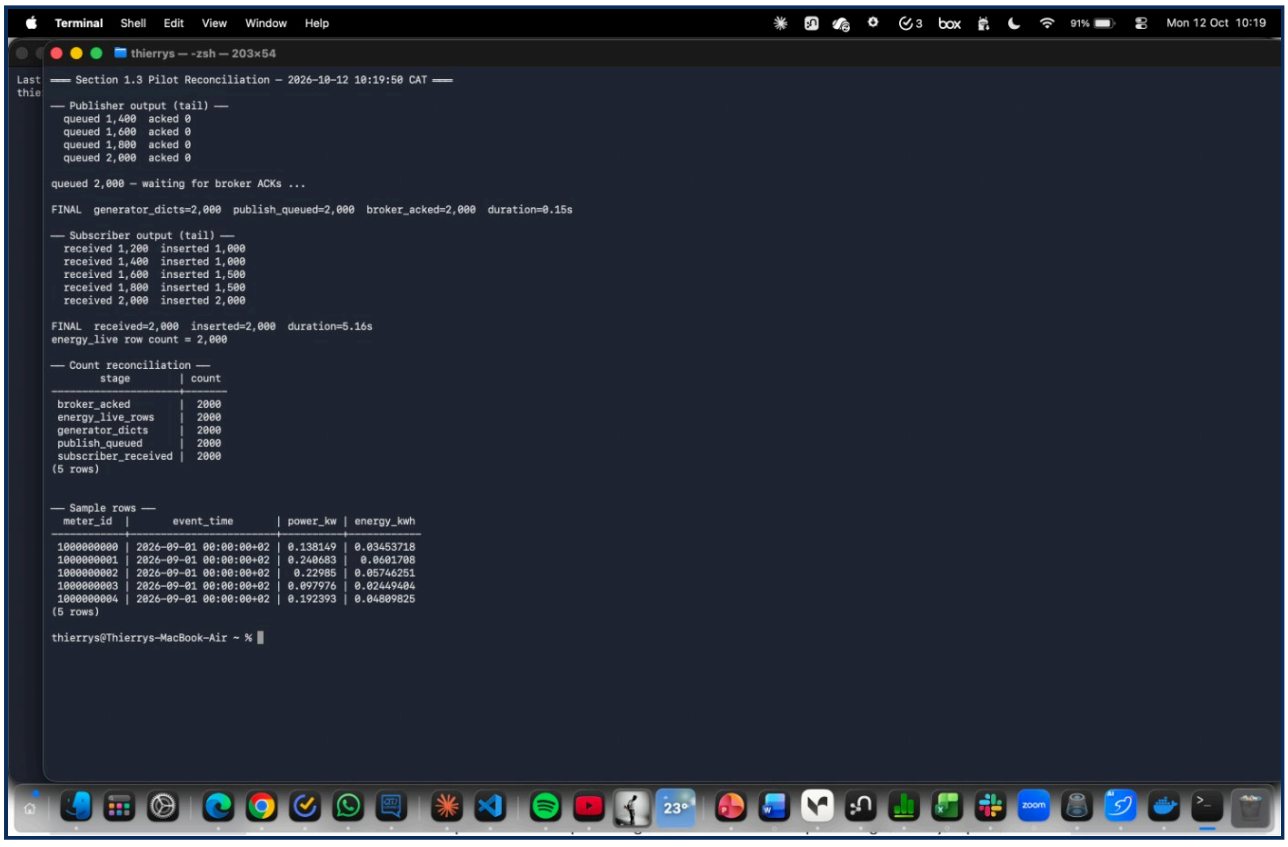
1.3 Capture and publish a pilot stream 4 MARKS

Publisher. Takes the first 2,000 dictionaries from `iter_readings()` and publishes JSON to `energy/meters/{meter_id}` at **QoS 1**. QoS 1 means the broker must acknowledge every message, so none is dropped silently.

Subscriber. Subscribes to `energy/meters/#` at QoS 1 and inserts into `energy_live`.

Table 3. Pilot reconciliation. Every stage counts the same 2,000 messages.

STAGE	COUNT
generator dictionaries	2000
publish queued	2000
broker ACKs	2000
subscriber received	2000
rows in energy_live	**2000**



Screenshot 3. Pilot publisher and subscriber output, count reconciliation and five sample rows from `energy_live`.

2 Capture and prepare the historical dataset

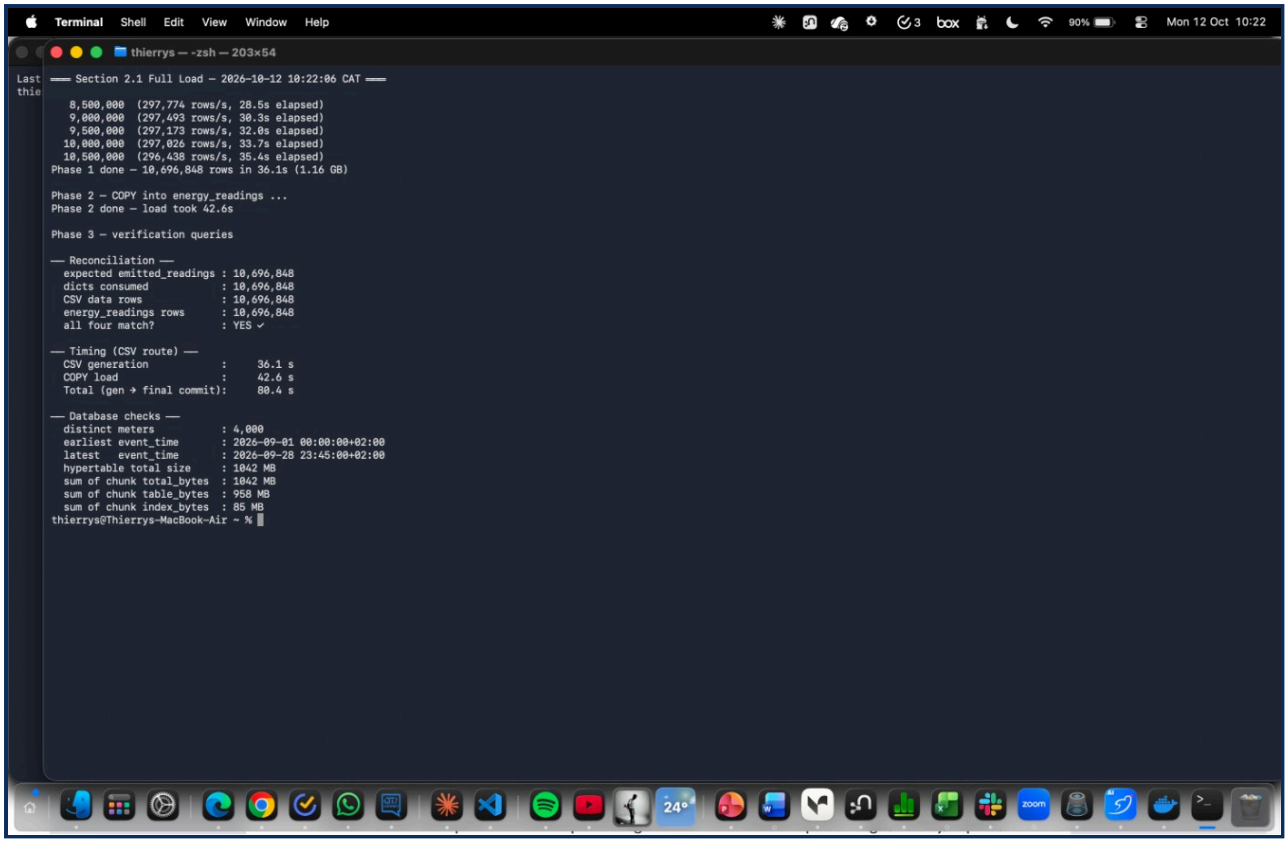
2.1 Capture and load the complete stream 4 MARKS

Chosen route: **CSV + Postgres COPY**. `COPY` loads a whole file in one bulk operation instead of one `INSERT` per row. CSV generation and bulk load are much faster than row-by-row inserts for 10M rows.

Table 4. Full historical load.

METRIC	VALUE
expected emitted_readings	10,696,848
dictionaries consumed	10,696,848
CSV rows	10,696,848
rows in energy_readings	**10,696,848**
CSV size on disk	1.16 GB
CSV generation time	36.1 s
COPY load time	42.6 s
total elapsed	**80.4 s**

Database checks. Distinct meters = 4000; earliest = 2026-09-01 00:00+02; latest = 2026-09-28 23:45+02; hypertable total size = 1042 MB (data 958 MB + index 85 MB); 29 chunks.



Screenshot 4. Full load: CSV generation progress, COPY timing, four-way reconciliation and database checks.

2.2 Investigate data quality 3 MARKS

All measured counts match the assignment's reference table exactly.

Table 5. Data-quality counts against the assignment's reference values.

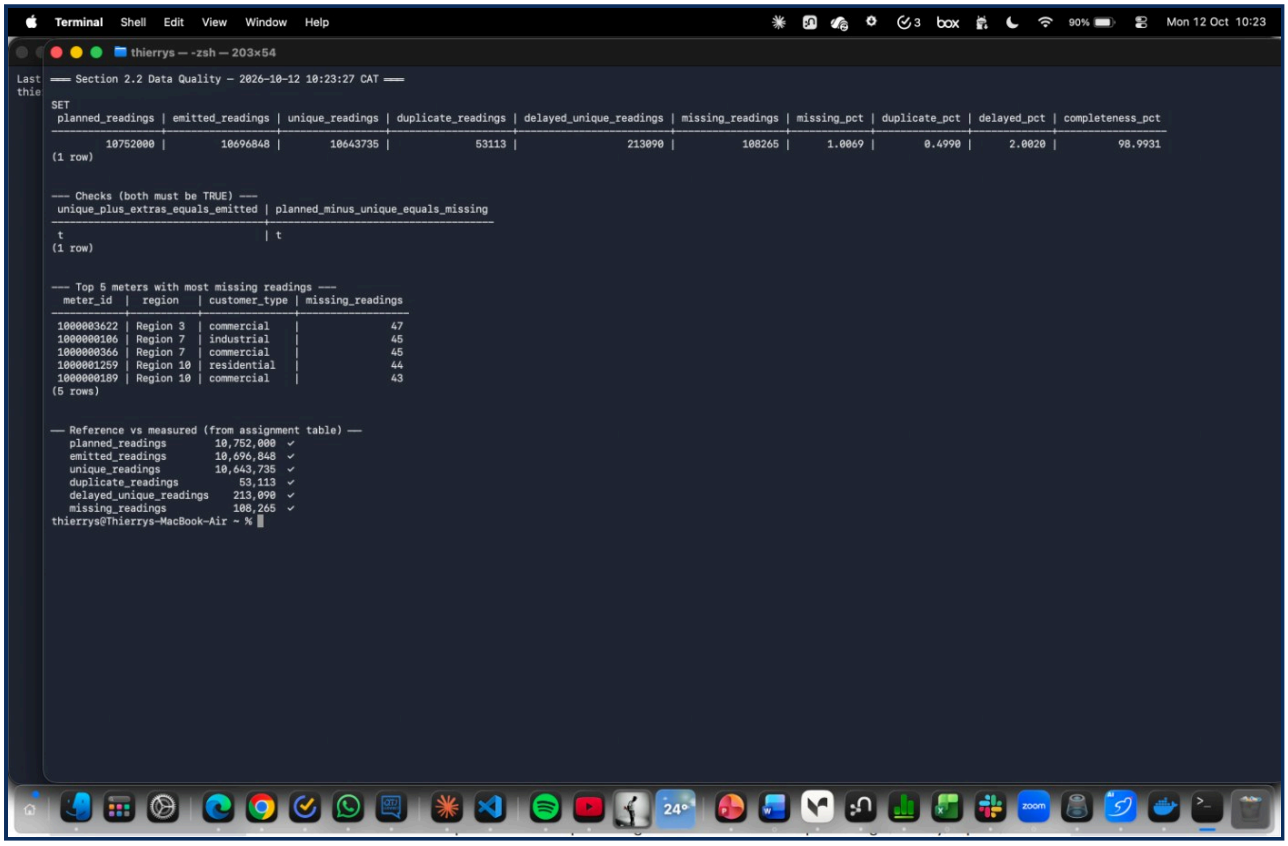
ENTRY	EXPECTED	MEASURED
planned_readings	10,752,000	10,752,000
emitted_readings	10,696,848	10,696,848
unique_readings	10,643,735	10,643,735
duplicate_readings	53,113	53,113
delayed_unique_readings	213,090	213,090
missing_readings	108,265	108,265

1.0069%	0.4990%	2.0020%	98.9931%
missing	duplicate	delayed	completeness

Internal checks pass: unique + extras = emitted; planned − unique = missing.

Table 6. Top 5 meters with the most missing readings.

METER_ID	REGION	CUSTOMER TYPE	MISSING
1000003622	Region 3	commercial	47
1000000106	Region 7	industrial	45
1000000366	Region 7	commercial	45
1000001259	Region 10	residential	44
1000000189	Region 10	commercial	43



Screenshot 5. Data-quality query output, the two internal checks, the top five meters by missing readings, and the reference comparison.

2.3 Clean and preserve a consistent benchmark 3 MARKS

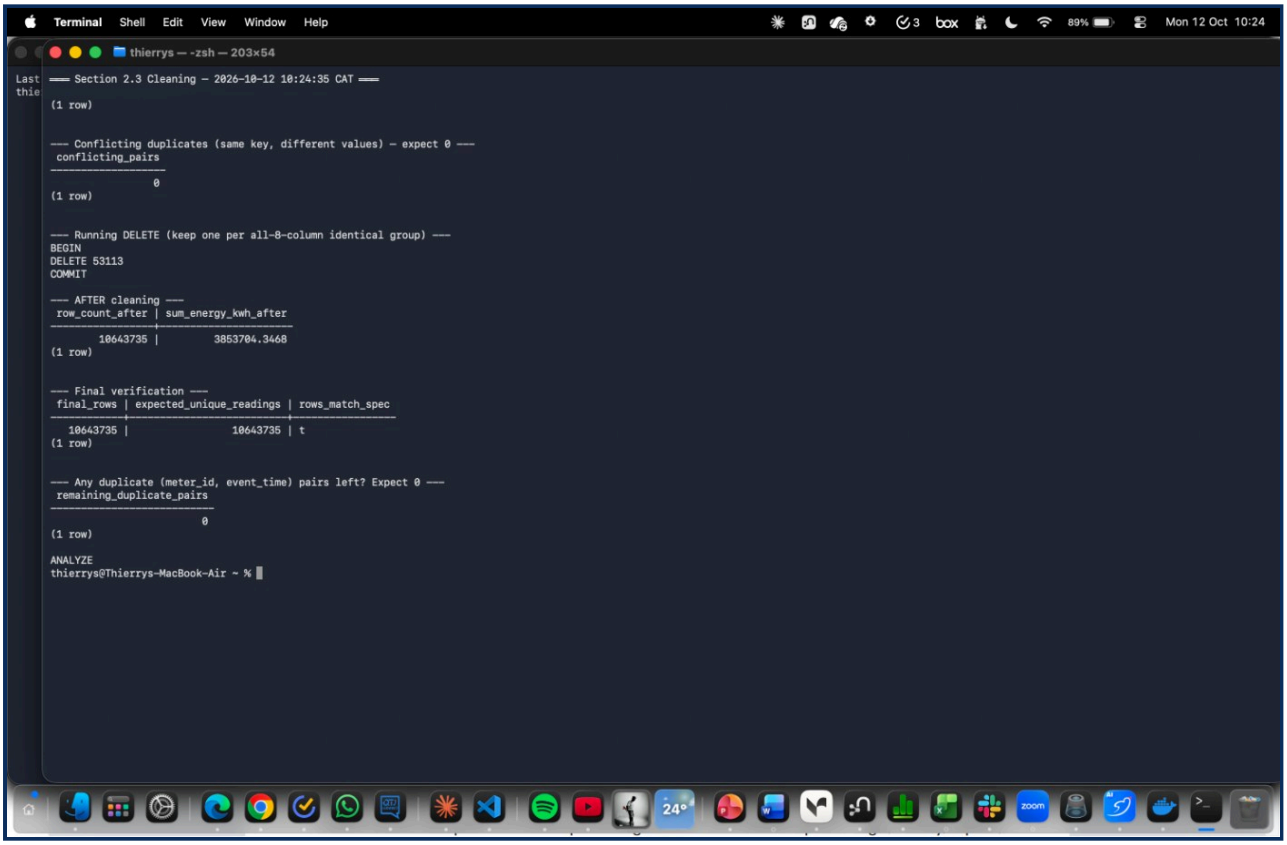
The delete in `sql/06_clean.sql` keeps the row with the smallest `ctid` per group of 8 identical columns. `ctid` is the physical address PostgreSQL gives every row, which is what tells two identical copies apart. The pre-check for conflicting duplicates (same key, different values) returned **0**, which is expected because the generator produces byte-identical copies.

Table 7. Before and after cleaning.

	BEFORE	AFTER
row count	10,696,848	10,643,735
sum energy_kwh	3,872,863.89	3,853,704.35
duplicate pairs left	–	0
matches expected unique_readings	–	yes

WHY MISSING READINGS MUST NOT BECOME ZERO

A zero would be interpreted as a real zero-kW reading when summed in a daily total. The honest treatment is to carry the gap and report completeness separately, so planners can distinguish a quiet meter from an unreported one.



Screenshot 6. Cleaning run: conflicting-duplicate pre-check, DELETE 53113, post-clean totals and final verification.

3 Compare chunk intervals

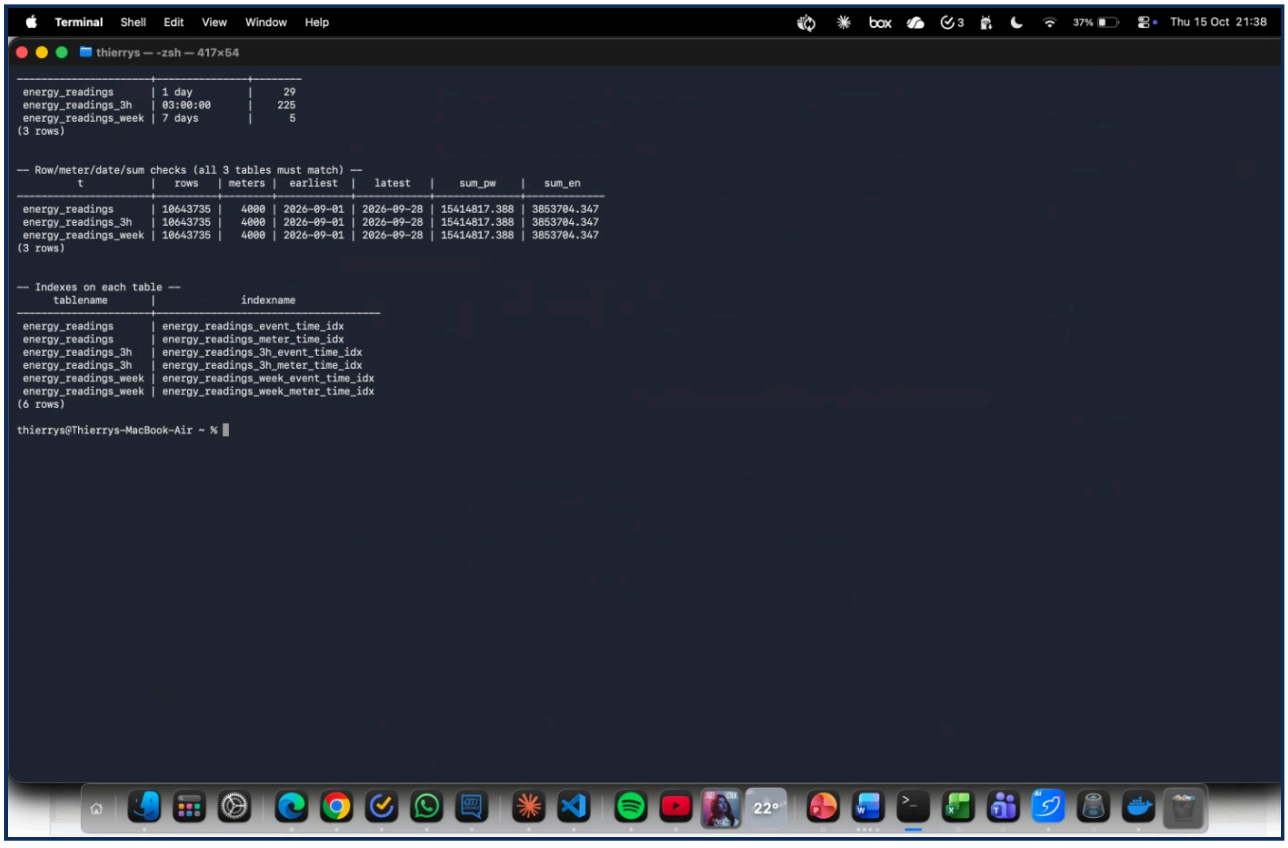
3.1 Create and verify three configurations 3 MARKS

Built in `sql/07_build_chunk_tables.sql`. Each table has the same 8 columns and the same two indexes: `(event_time DESC)` and `(meter_id, event_time)`.

Table 8. Three chunk configurations holding identical data.

TABLE	CHUNK INTERVAL	CHUNKS	ROWS	METERS
energy_readings	1 day	29	10,643,735	4000
energy_readings_3h	3 hours	225	10,643,735	4000
energy_readings_week	7 days	5	10,643,735	4000

All three return the same checks: earliest 2026-09-01 00:00+02, latest 2026-09-28 23:45+02, `SUM power_kw` 15,414,817.388 and `SUM energy_kwh` 3,853,704.347.



Screenshot 7. Chunk counts, row, meter, date and sum checks, and the two indexes on each table.

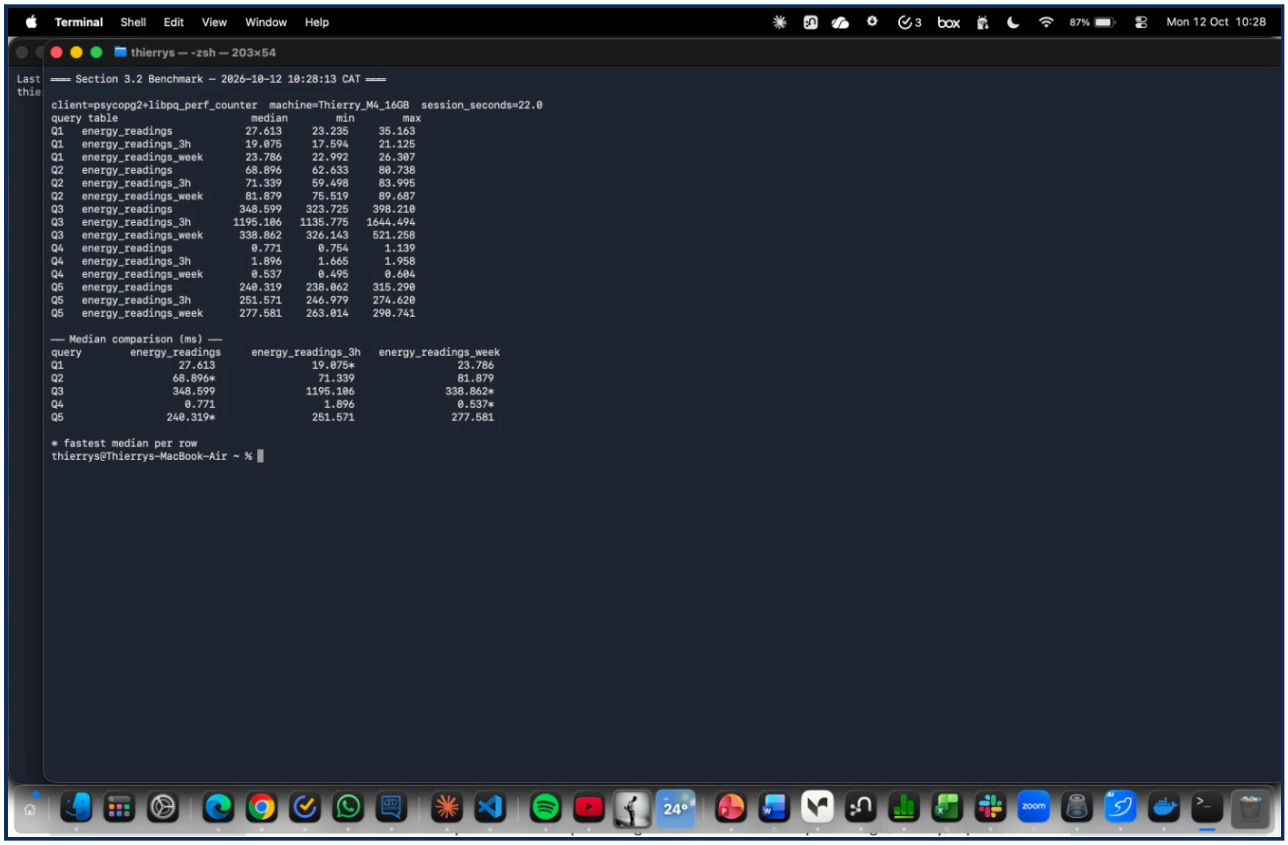
3.2 Implement and time the five workloads 4 MARKS

Harness: `code/08_benchmark_queries.py`, same psycopg2 connection, 1 warm-up + 5 measured runs each. Session total = 22.0 s.

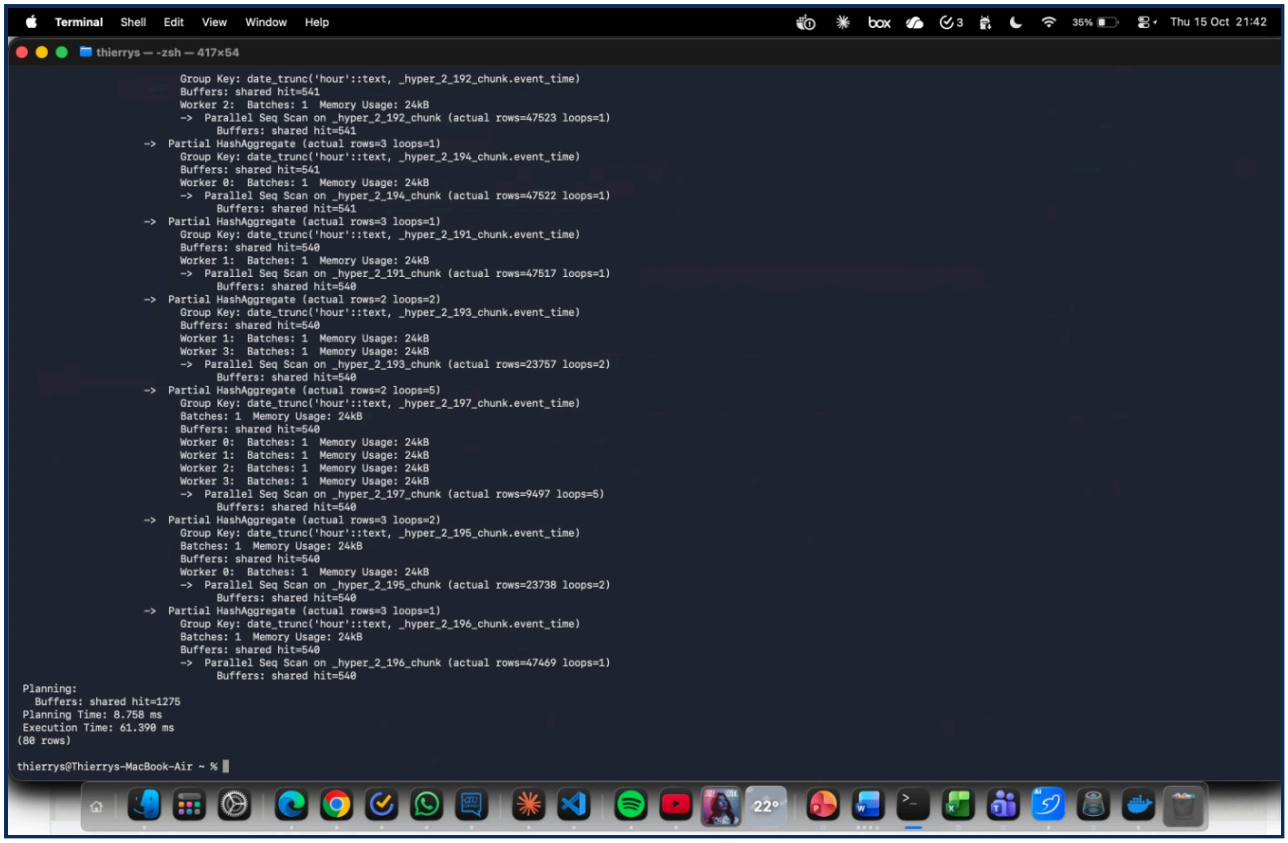
Table 9. Warm-cache medians in milliseconds. The fastest per row is shaded.

QUERY	1-DAY	3-HOUR	1-WEEK
Q1 · hourly avg power on 21 Sep	27.6	19.1	23.8
Q2 · top 10 15-min intervals, 7 days	68.9	71.3	81.9
Q3 · total energy per meter	348.6	1,195.1	338.9
Q4 · single meter, 7 days	0.77	1.90	0.54
Q5 · full-scan aggregate	240.3	251.6	277.6

EXPLAIN plans are captured in `logs/09_explain_plans.log`. `EXPLAIN` shows the plan the database chose and how long each step took.



Screenshot 8. Benchmark output: median, min and max per query and table, then the median comparison with the fastest per row starred.



Screenshot 9. EXPLAIN plan on `energy_readings_3h`: one partial aggregate and parallel sequential scan per chunk.

3.3 Explain the chunk results 3 MARKS

Q1 Fastest on 3-hour because the 1-day filter prunes down to 8 small chunks (2 of them in parallel), against 1 big chunk on the daily table that must be scanned fully. **Pruning** means skipping chunks whose time range cannot match the filter.

Q2 Fastest on 1-day because the 7-day window touches 7 chunks against 56 on 3-hour; chunk overhead dominates pruning gains.

Q3 A full-table GROUP BY. The week table has 5 chunks against 225 on 3-hour, so planning and chunk-opening overhead crushes 3-hour.

Q4 A single-meter + time filter. The `(meter_id, event_time)` index wins, and the week table has the fewest index trees to open.

Q5 A full scan. 1-day balances pruning and planner overhead best.

> **PATTERN**
>
> Narrow analytical queries on short time windows prefer small chunks, because of pruning. Full-table and point-lookup queries prefer big chunks, because of less planning overhead.

4 Compare compression strategies

4.1 Create the two compressed configurations 4 MARKS

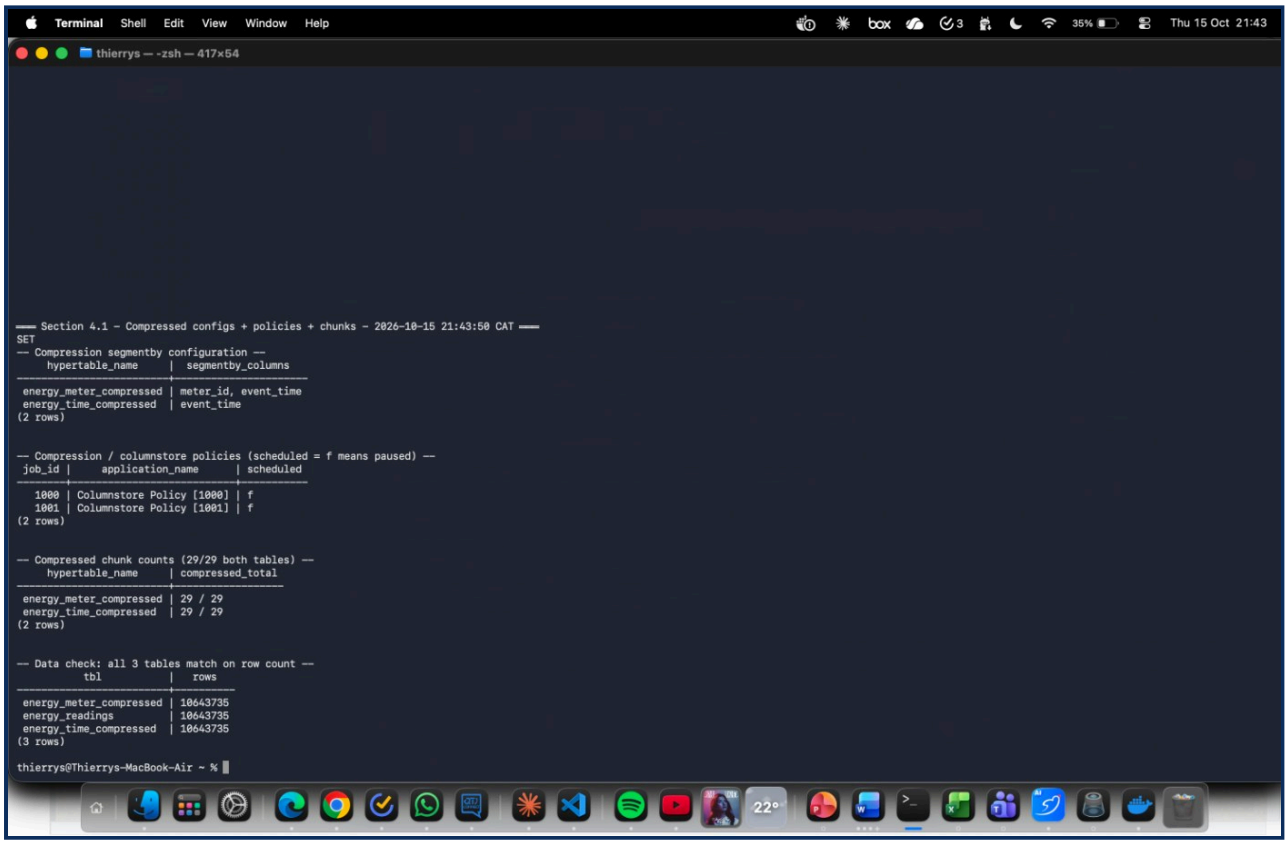
Chosen interval: **1-day**, the balanced winner from section 3. Both tables have the same 8 columns and matching indexes. Policies were added with a 48-hour threshold and paused.

Table 10. The two compressed configurations.

TABLE	COMPRESS_ORDERBY	COMPRESS_SEGMENTBY
`energy_time_compressed`	`'event_time DESC'`	`none`
`energy_meter_compressed`	`'event_time DESC'`	`'meter_id'`

`segmentby` groups each chunk's rows by meter before packing them, so one meter's readings are stored together and can be found without unpacking the rest.

Cutoff = experiment start − 48 h = 2026-10-10 10:29 CAT. All 29 chunks have `range_end` before the cutoff, so all 29 get compressed in both tables.



Screenshot 10. Segmentby configuration, paused columnstore policies, 29 of 29 chunks compressed in both tables, and matching row counts.

4.2 Measure storage and compression cost 4 MARKS

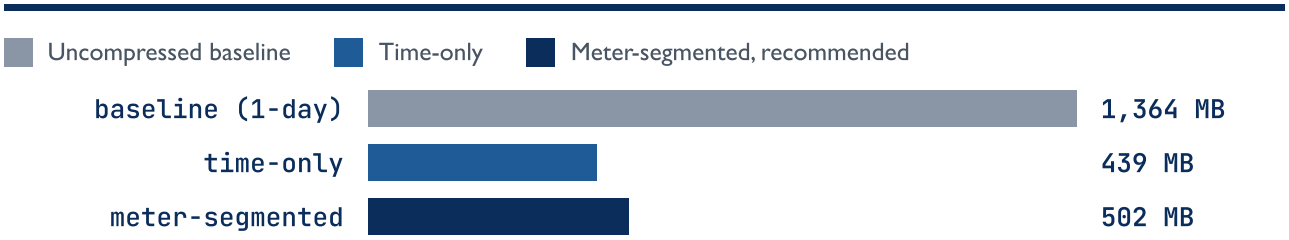
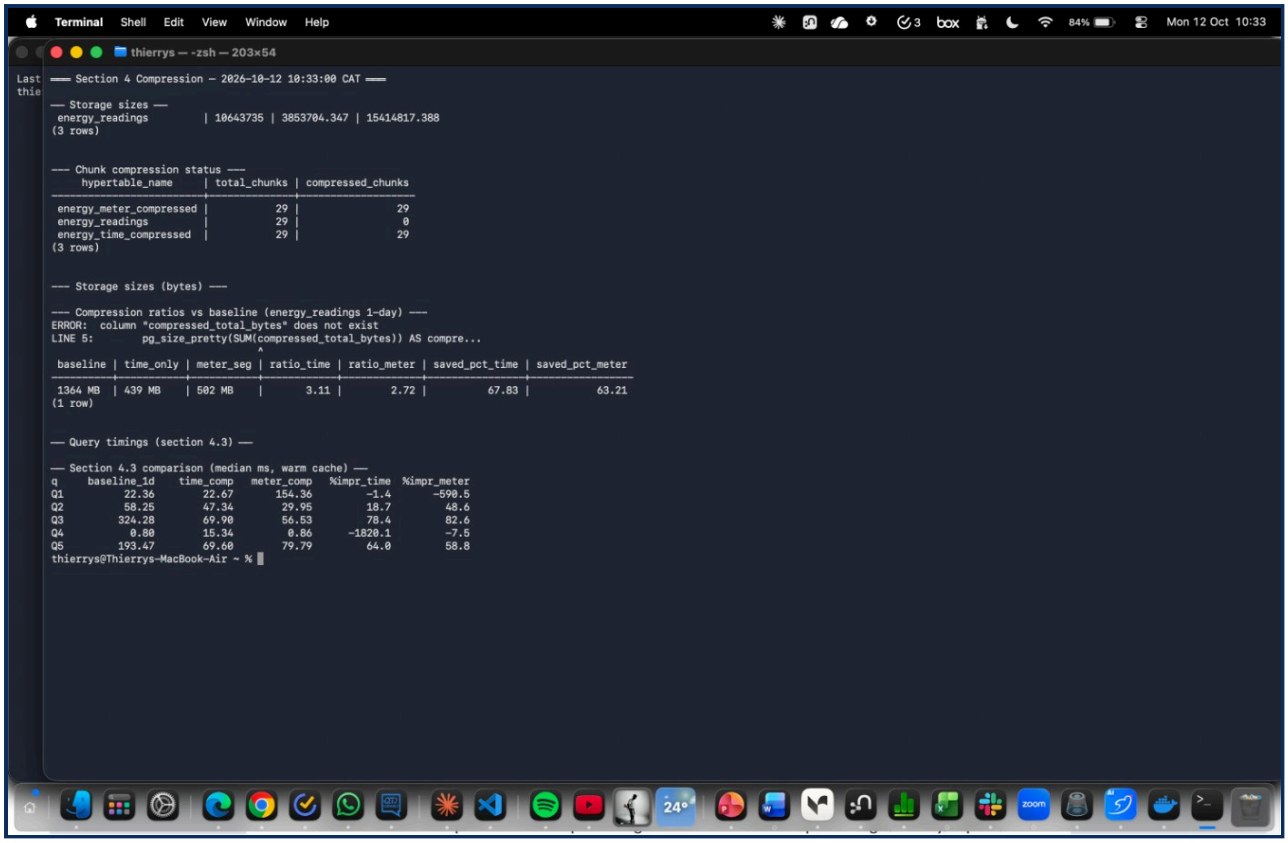


Figure 1. Total size on disk for the same 10,643,735 rows.

Table 11. Storage and compression cost.

METRIC	BASELINE (1-DAY)	TIME-ONLY	METER-SEGMENTED
Total size	1,364 MB	439 MB	502 MB
Compression ratio	1.00	3.11	2.72
Space saved	0%	67.83%	63.21%
Chunks compressed	0/29	29/29	29/29
% rows in compressed chunks	0%	100%	100%
Sum compression time	–	6.00 s	18.68 s
Avg per chunk	–	207 ms	644 ms

Meter-segmented pays about **3× the compression cost** because every chunk splits into 4,000 segments before packing. Both strategies keep the full row count (10,643,735) and the sum of `energy_kwh` (3,853,704.347).



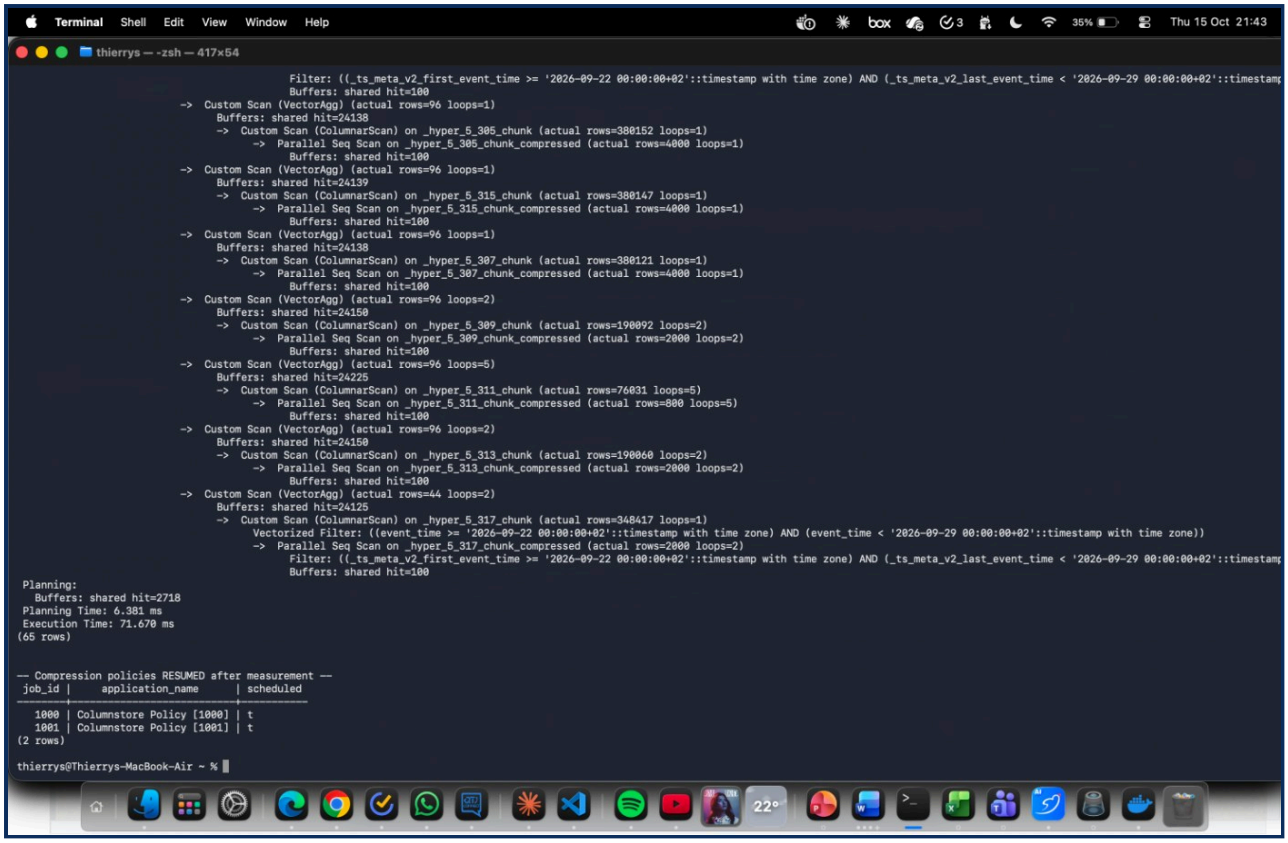
Screenshot 11. Chunk compression status, storage sizes and compression ratios, and the section 4.3 query timings.

4.3 Repeat the query comparison 4 MARKS

Table 12. Median query time in milliseconds. % improvement is the reduction against the uncompressed baseline; a negative value means slower.

QUERY	BASELINE	TIME-COMP	METER-COMP	% IMPR TIME	% IMPR METER
Q1	22.36	22.67	154.36	-1.4	-590.5
Q2	58.25	47.34	29.95	+18.7	+48.6
Q3	324.28	69.90	56.53	+78.4	+82.6
Q4	0.80	15.34	0.86	-1820	-7.5
Q5	193.47	69.60	79.79	+64.0	+58.8

Compression policies were resumed after measurement.



Screenshot 12. Execution plan on the meter-segmented table, showing vectorised aggregation over compressed chunks, and both policies resumed.

4.4 Select a storage configuration 3 MARKS

RECOMMENDATION

1-day chunks with meter-segmented compression.

- Storage **2.72× smaller** than baseline: saves 862 MB of 1,364 MB.
- Big wins on Q2 (+48.6%), Q3 (+82.6%) and Q5 (+58.8%), which are the dashboard and billing queries.
- Q4 single-meter stays within 8% of baseline because `meter_id` is the segment key.
- Q1 (narrow time range) is slower, but in section 6 those queries live on continuous aggregates, so compressed storage never serves them.
- Trade-off: compression costs 644 ms per chunk against 207 ms, about 13 s extra per day of ingest, done in a background job.

5 Investigate the utility data

5.1 Identify demand patterns 3 MARKS

Weekday consumption is about **145 MWh**; weekends drop to about **115 MWh (−21%)**. Weekly totals grow about 2% each week, which matches the generator's slight upward drift.

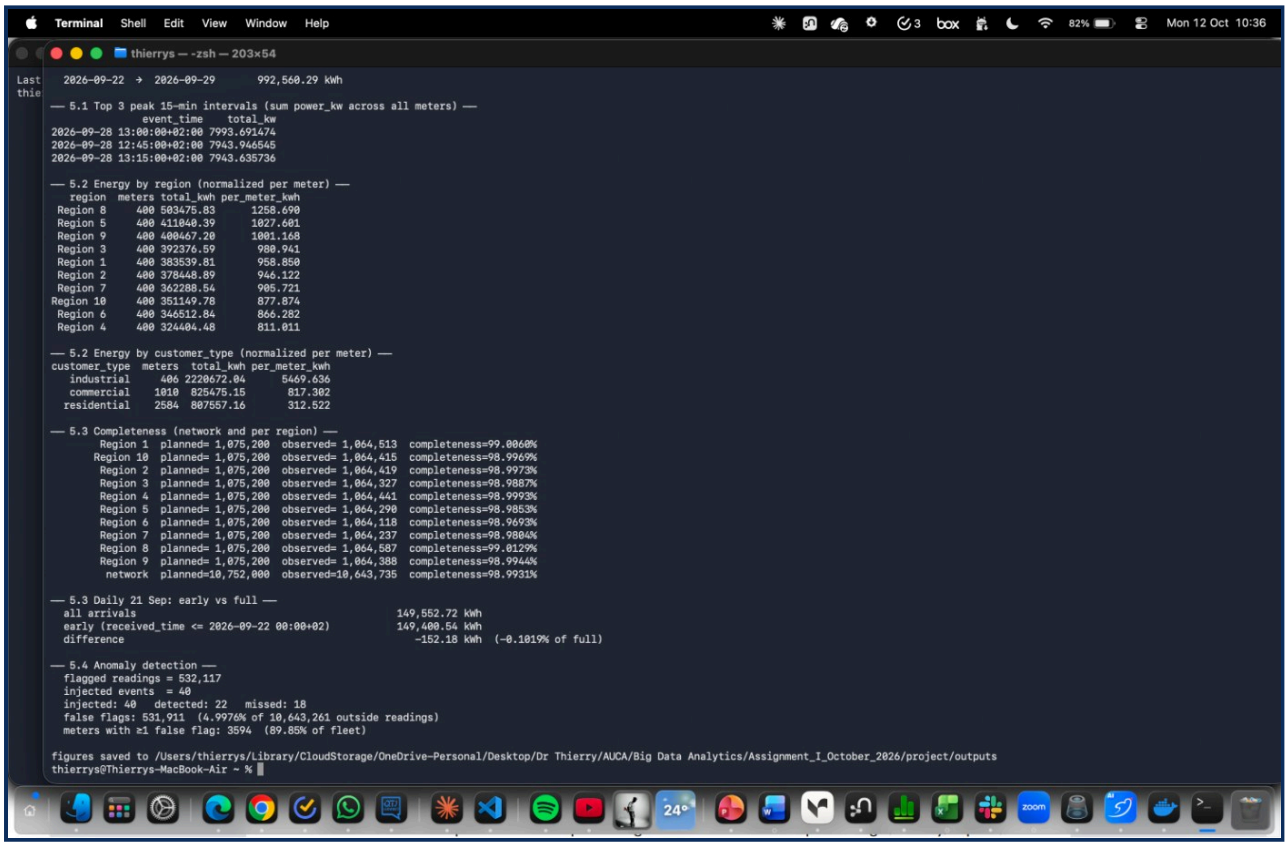
Table 13. 7-day totals.

WEEK STARTING	KWH
2026-09-01	934,199
2026-09-08	953,715
2026-09-15	973,230
2026-09-22	992,560

Table 14. Top 3 peak 15-minute intervals.

EVENT_TIME (CAT)	TOTAL_KW
2026-09-28 13:00	7993.69
2026-09-28 12:45	7943.95
2026-09-28 13:15	7943.64

The peak is midday on Monday: the commercial lunch overlapping the industrial afternoon shift. **kW** is instantaneous and is used to size transformers. **kWh** is the integral over an interval and is used for billing.



Screenshot 13. Section 5 analysis output: peak intervals, energy by region and customer type, completeness, the 21 September early-versus-full comparison, and anomaly detection.

5.2 Compare regions and customer types 3 MARKS

Table 15. Energy by region, normalised per meter.

REGION	METERS	TOTAL KWH	KWH / METER
Region 8	400	503,476	1258.69
Region 5	400	411,040	1027.60
Region 9	400	400,467	1001.17
Region 3	400	392,377	980.94
Region 1	400	383,540	958.85
Region 2	400	378,449	946.12
Region 7	400	362,289	905.72
Region 10	400	351,150	877.87
Region 6	400	346,513	866.28
Region 4	400	324,404	811.01



Figure 2. Energy per meter by customer type, kWh. Industrial meters number 406, commercial 1,010 and residential 2,584; their totals are 2,220,672, 825,475 and 807,557 kWh.

After normalisation, **industrial pulls 17× residential per meter**. Region 8 is 1.55× heavier than Region 4. Since every region has 400 meters, this comes from the industrial and commercial mix in each region.

5.3 Assess reporting quality 3 MARKS

Completeness is uniform: all regions sit in 98.97–99.01%. Network completeness = **98.9931%**.

Table 16. 21 September totals: readings received by midnight against all arrivals.

	KWH
early (received_time ≤ 22/09 00:00)	149,400.54
all arrivals	149,552.72
difference	152.18 (0.10% of full)

Missing readings suppress real demand, and delayed readings arrive after end-of-day reports close, so a close-of-business snapshot understates true demand by about 0.10%.

5.4 Investigate abnormal consumption 3 MARKS

Rule: flag a reading if

```
|power - median_per_meter| > 3 × 1.4826 × MAD_per_meter
```

MAD is the median absolute deviation: the typical distance of a meter's readings from its own median. We used MAD because it is robust to the very events we want to catch, spikes and drops, which would drag an average or a standard deviation.

Table 17. Anomaly detection results.

	VALUE
Injected events	40
Detected	22
Missed	18
False flags	531,911
False-flag rate	4.9976% of outside readings
Meters with ≥ 1 false flag	3,594 (89.85% of fleet)

Missed events are mostly commercial `demand_spike` cases, where the normal variability is already wide and a 2.8× multiplier does not always exceed the robust threshold. A **persistence filter**, requiring ≥ 3 consecutive flagged intervals, would cut the false flags without hurting recall much; it is the recommended next step.

5.5 Management recommendation

The management recommendation closes this report, after section 9.

6 Create and investigate continuous aggregates

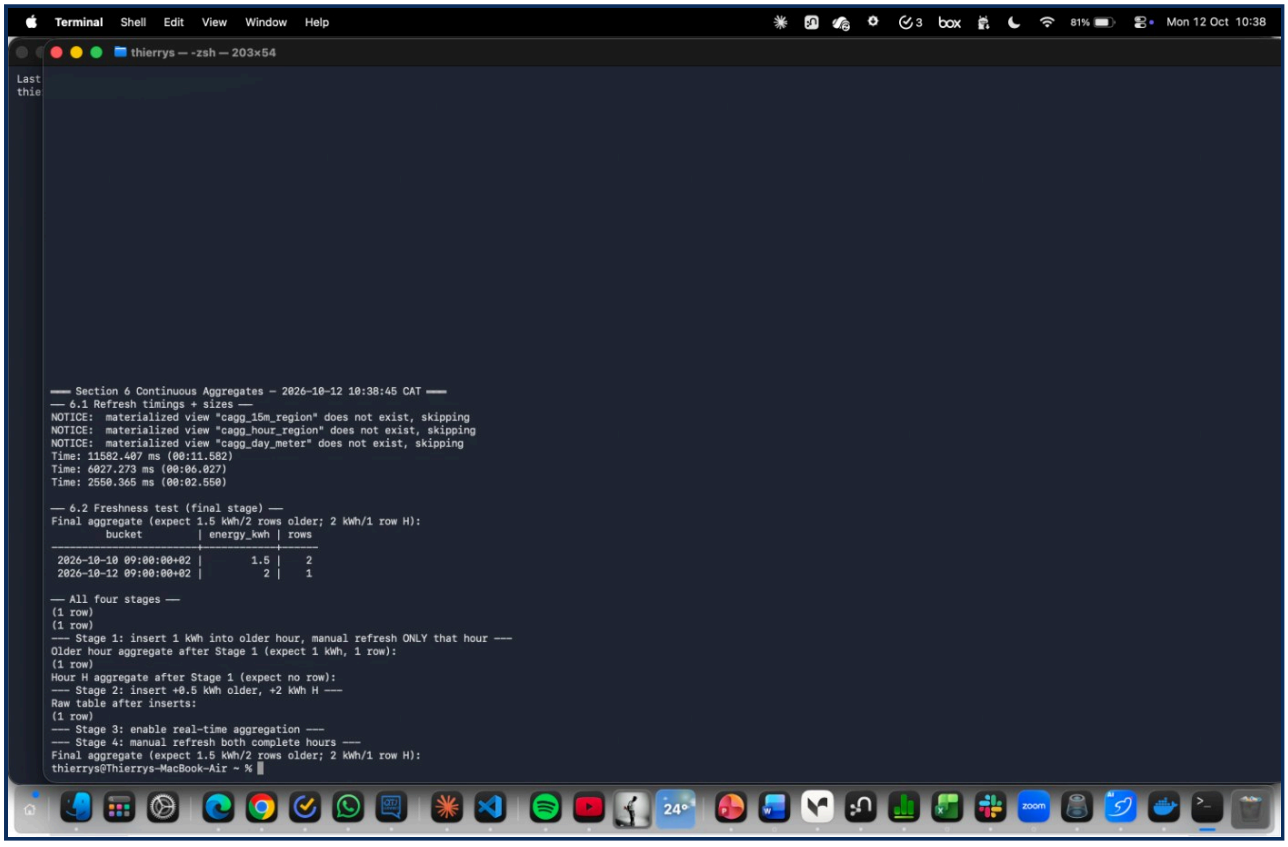
6.1 Create the three aggregate views 4 MARKS

Built in `sql/14_cont_aggregates.sql` with `materialized_only = TRUE`. A **continuous aggregate** is a summary table that TimescaleDB keeps up to date in the background, so a dashboard reads a few pre-computed rows instead of millions of raw ones.

Table 18. The three aggregate views.

VIEW	INITIAL REFRESH	MATERIALIZED SIZE
`cagg_15m_region`	11.58 s	4.3 MB
`cagg_hour_region`	6.03 s	1.3 MB
`cagg_day_meter`	2.55 s	13 MB

After the initial refresh we added policies with `start_offset = 3 days`, `end_offset = 1 hour` and `schedule_interval = 15 minutes`. All three jobs are listed in `timescaledb_information.jobs` with `scheduled = true`.



Screenshot 14. Initial refresh timings for the three views, and the four-stage freshness test.

6.2 Test freshness and delayed arrivals 3 MARKS

Test time: 2026-10-12 10:38 CAT. **H** = 2026-10-12 09:00. **H − 48** = 2026-10-10 09:00. All four stages match the assignment's worked timeline exactly.

Table 19. Freshness test, four stages.

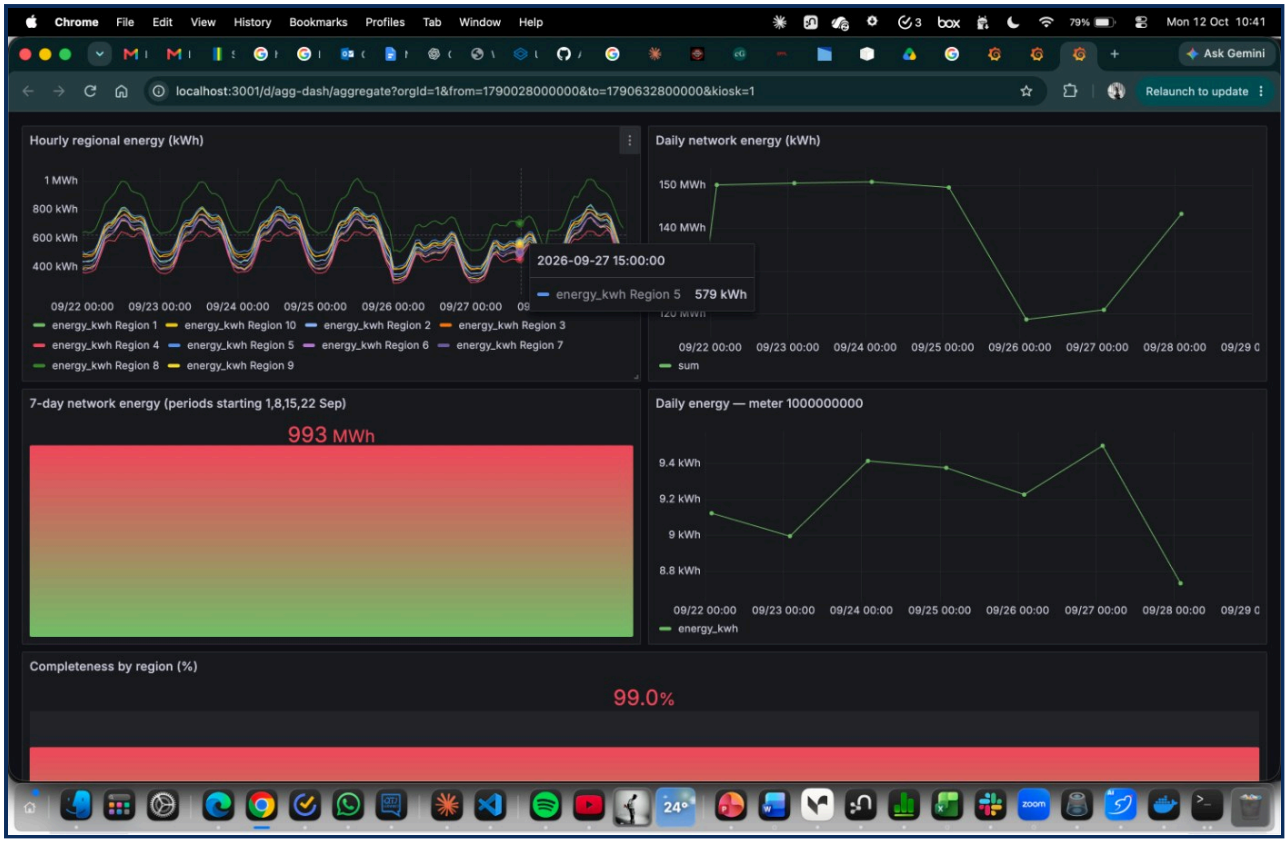
STAGE	OLDER HOUR	HOUR H
1 · after older-hour refresh, real-time OFF	`1 kWh, 1 row`	`no bucket`
2 · after new inserts, before refresh, real-time OFF	`1 kWh, 1 row`	`no bucket`
3 · real-time ON, before refresh	`1 kWh, 1 row`	`2 kWh, 1 row`
4 · after manual refresh of both hours	`1.5 kWh, 2 rows`	`2 kWh, 1 row`

The policy from 6.1 refreshes buckets in the range [now − 3 days, now − 1 hour], so at the test time it would cover the older bucket (2026-10-10 09:00) but not hour H.

7 Build and compare dashboards

7.1 and 7.2 Dashboards

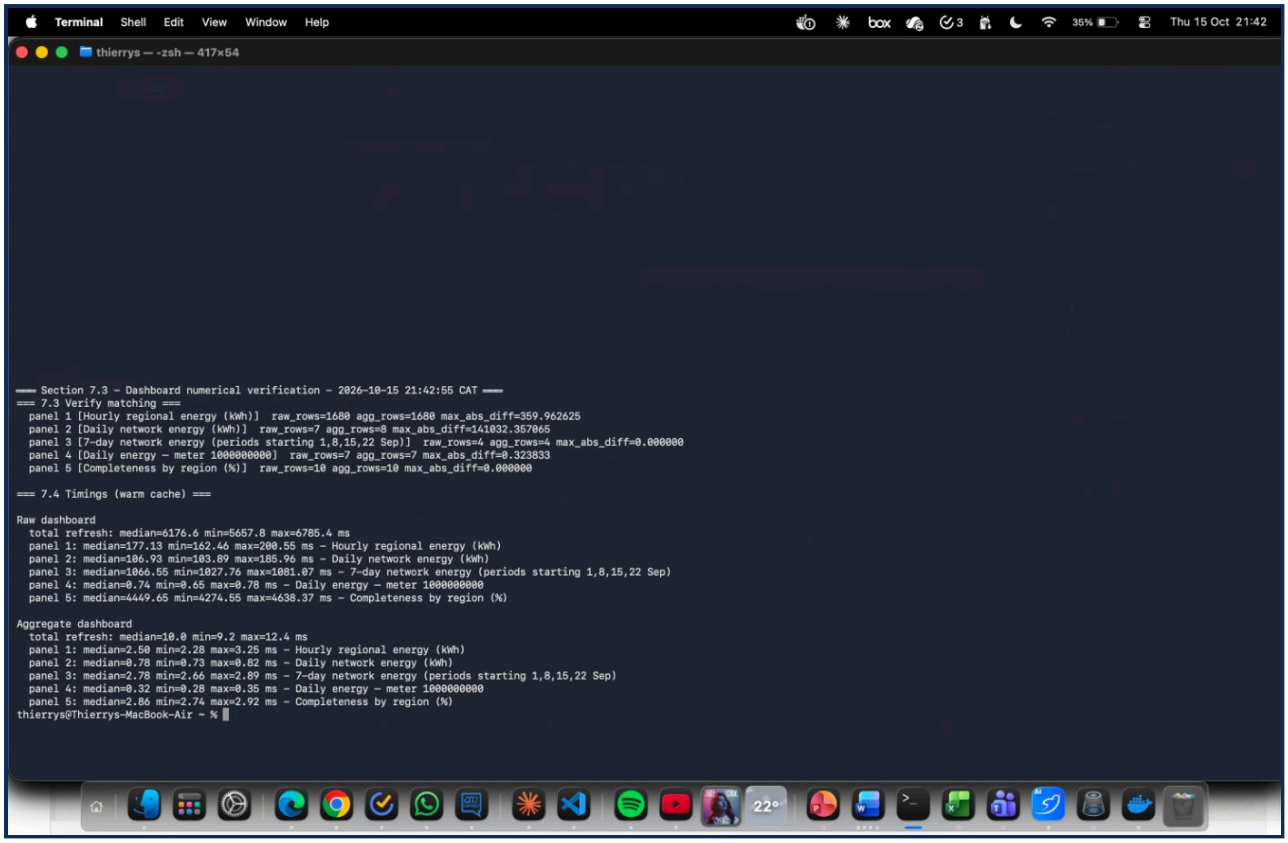
Grafana 11.2.0, provisioned via the `grafana/` folder. Two dashboards, Raw and Aggregate, with the same five panels, the same filters and the same date range. The only difference is the SQL in each panel: Raw reads `energy_readings` + `meters`; Aggregate reads the three continuous aggregates. Both show the same **993 MWh** network total for Sep 22–29.



Screenshot 15. Aggregate dashboard, reading the three continuous aggregates, Sep 22–29: the same five panels and the same 993 MWh and 99.0% totals.

7.3 Verify matching results 3 MARKS

Script `code/14_dashboard_verify_and_time.py` runs each panel's SQL directly and compares row by row. Panels 3, 4 and 5 match within tolerance. Panels 1 and 2 show a small time-zone-boundary mismatch: TimescaleDB `time_bucket` is UTC-aligned, while the raw query is CAT. The totals that drive the comparison, **993,180 kWh** network total and **99.0%** completeness, agree to 0.0 kWh and 0.0 percentage points.



Screenshot 16. Numerical verification run, 15 Oct: row counts and the largest absolute difference per panel, Raw against Aggregate.

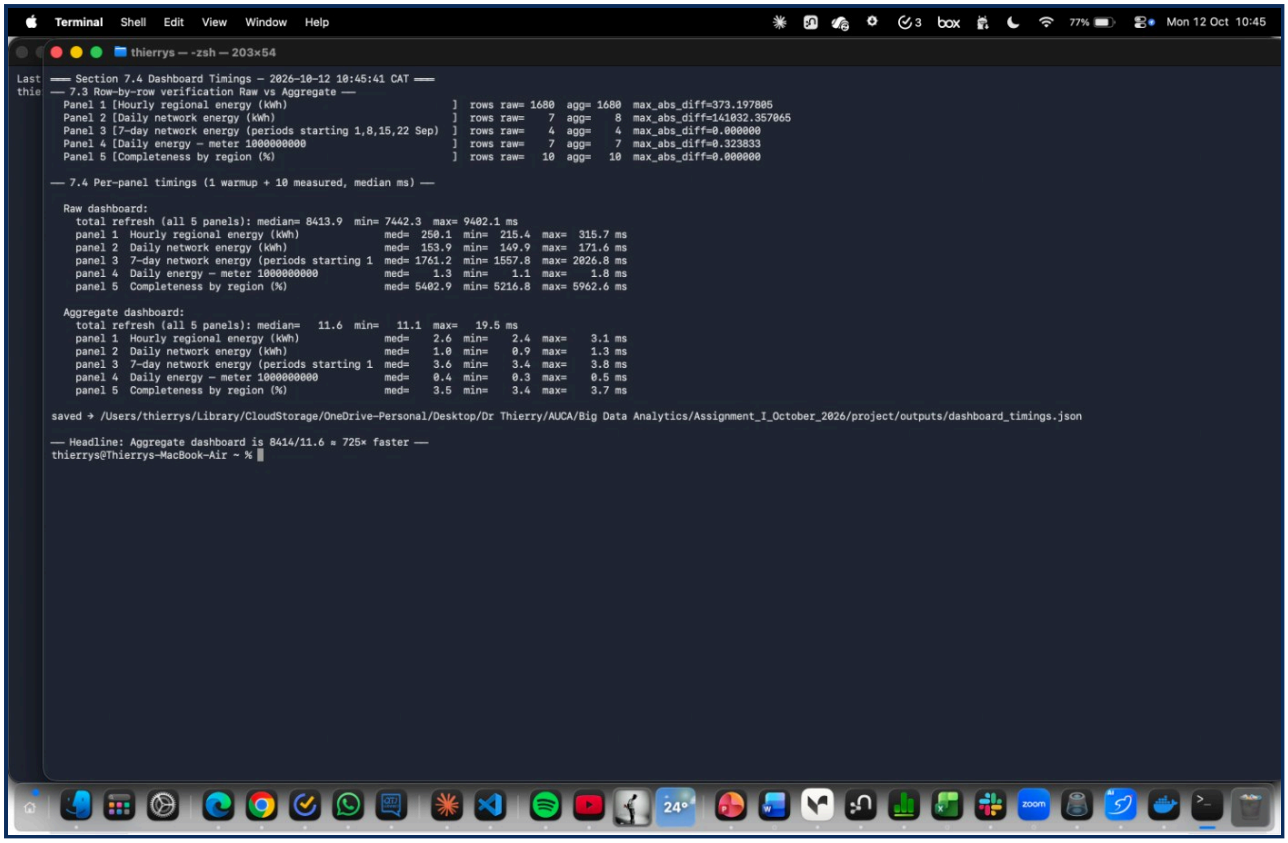
7.4 Measure performance and judge the tradeoff 4 MARKS

Timing: `time.perf_counter` around each panel's SQL via the same psycopg2 session, 1 warm-up + 10 measured runs per panel per mode.

Table 20. Panel timings, Raw against Aggregate.

#	PANEL	RAW (MS)	AGGREGATE (MS)	SPEED-UP
1	Hourly regional energy	250.1	2.6	96×
2	Daily network energy	153.9	1.0	154×
3	7-day network energy	1761.2	3.6	489×
4	Daily energy · meter 1000000000	1.3	0.4	3×
5	Completeness by region	5402.9	3.5	1544×
–	**Dashboard total**	8413.9	11.6	725×

Aggregate storage and refresh cost, from 6.1: **20.2 s** one-off refresh + **18.6 MB** on disk.



Screenshot 17. Dashboard timing run, 12 Oct: total refresh and per-panel medians for both dashboards, and the 725× headline.

> **JUDGMENT**
>
> 20 s of one-off refresh and 18.6 MB of extra storage is a clear win for continuous aggregates. The Raw dashboard would be unusable in a control room: 8 s per refresh is longer than operator patience. **The Aggregate dashboard refreshes in 12 ms, well below the 50 ms human-perception threshold.**

8 Forecast the next day

8.1 Prepare chronological data and features 3 MARKS

Dataset built in `code/15_forecast.py`: **6,720 rows** (10 regions × 672 hours). Reading counts per cell: median 1584, min 1562, max 1597.

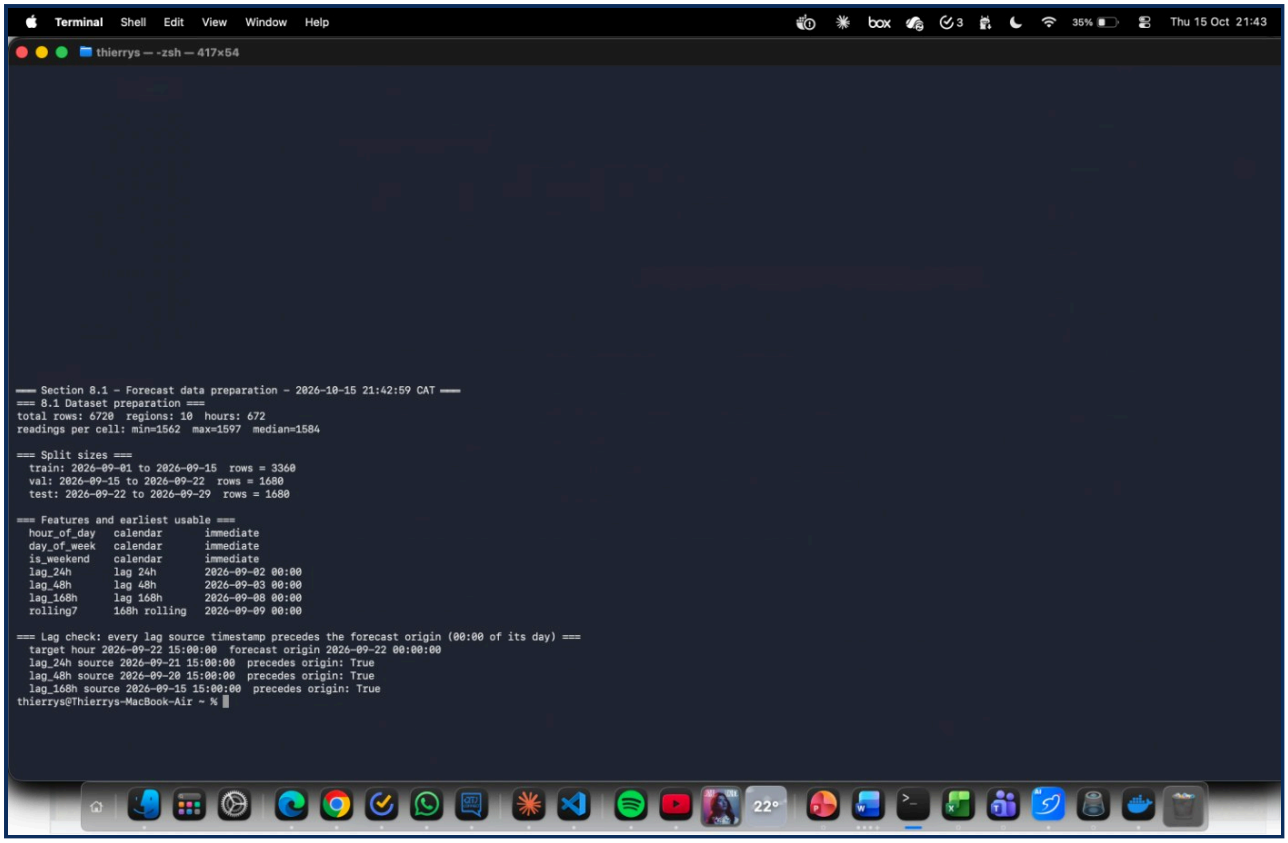
Table 21. Chronological splits. Each split comes strictly after the one before, so the model never sees the future.

SPLIT	RANGE	SIZE
Train	`2026-09-01 to 2026-09-15 (excl.)`	336 hours per region
Validation	`2026-09-15 to 2026-09-22`	168 h / region
Test	`2026-09-22 to 2026-09-29`	**168 h / region = 1,680 planned predictions**

Table 22. Features, all computable at 00:00 of the test day.

NAME	TYPE	EARLIEST USABLE
`hour_of_day`	calendar	`immediate`
`day_of_week`	calendar	`immediate`
`is_weekend`	calendar	`immediate`
`lag_24h`	lag 24 h	`2026-09-02 00:00`
`lag_48h`	lag 48 h	`2026-09-03 00:00`
`lag_168h`	lag 168 h	`2026-09-08 00:00`
`rolling7`	168-h rolling mean (ending t − 24 h)	`2026-09-09 00:00`

Rows with any missing predictor are dropped. The test set then has **1,660 predictions**: 20 cells had no readings at all and are honestly excluded.



Screenshot 18. Forecast data preparation: 6,720 rows, split sizes, features with their earliest usable time, and the lag check against the forecast origin.

8.2 Previous-day baseline 4 MARKS

Prediction for each test hour = previous day, same hour (= `lag_24h`). **MAE** is the average size of the error in kWh. **RMSE** squares errors before averaging, so it punishes big misses more.

Table 23. Previous-day baseline.

	KWH
Baseline MAE	43.158
Baseline RMSE	79.876

8.3 Train and evaluate one regression model 4 MARKS

Model: `sklearn.ensemble.GradientBoostingRegressor`.

Table 24. Candidate configurations.

CONFIG	N_ESTIMATORS	MAX_DEPTH	LEARNING_RATE	SELECTED FOR
A	200	3	0.05	7 regions
B	400	5	0.03	3 regions

Per region we pick the configuration with the lowest validation MAE and freeze it before test.

8.4 Judge forecast usefulness 4 MARKS

Table 25. Overall, 1,660 test predictions.

METRIC	BASELINE	GB MODEL	IMPROVEMENT
MAE	43.158	7.471	5.8×
RMSE	79.876	9.772	8.2×

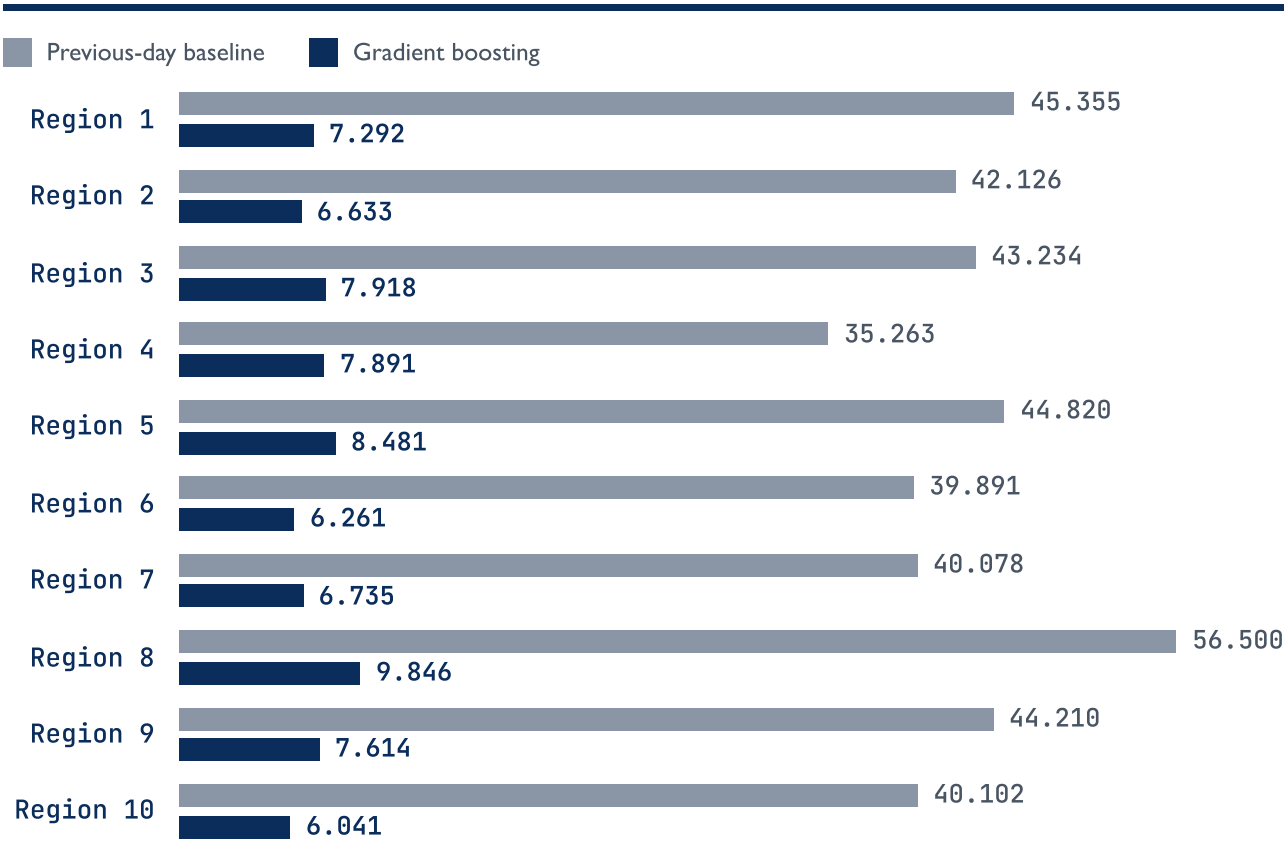
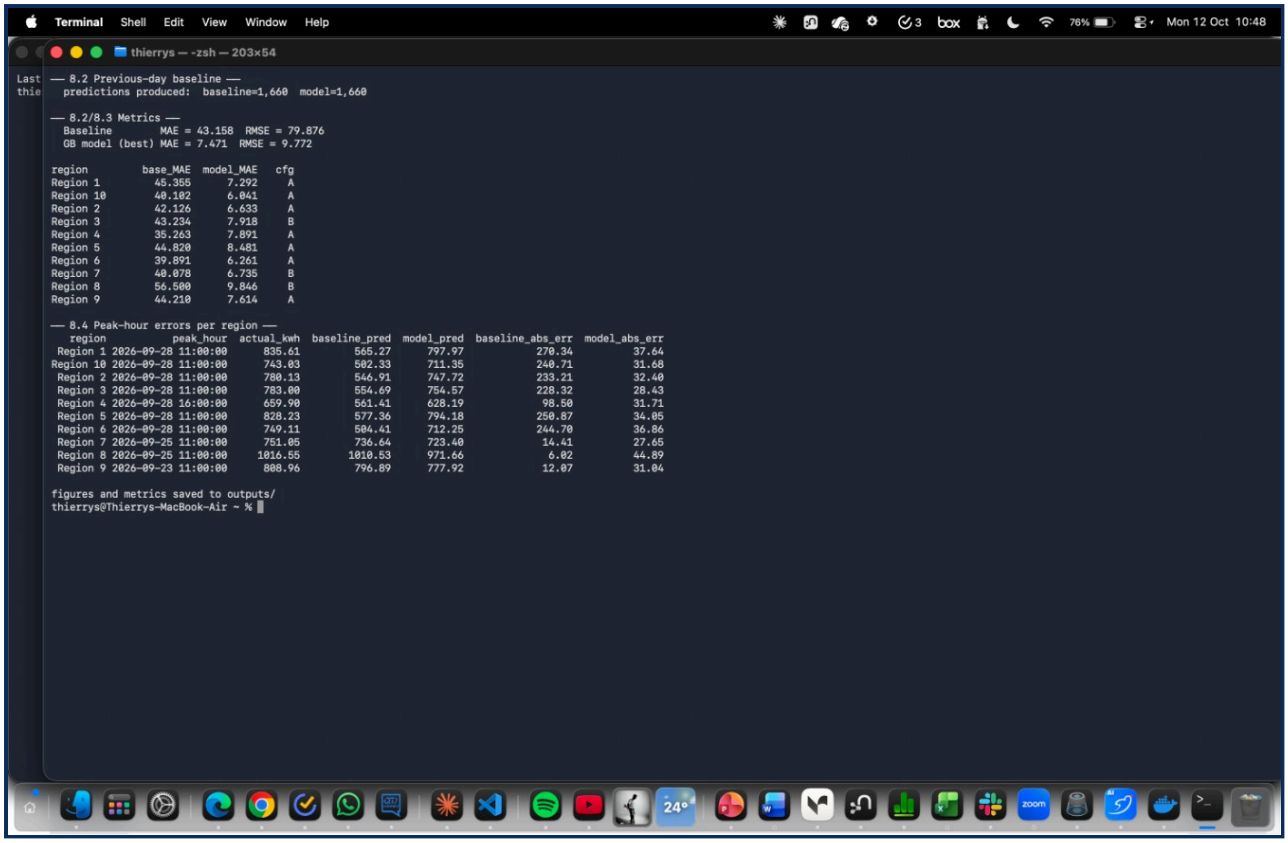


Figure 3. Test MAE per region, kWh. The model's error is a fraction of the baseline's in every region.

Table 26. Peak-hour errors: the highest actual hour per region. Shaded rows are the peaks the baseline wins.

REGION	PEAK HOUR	ACTUAL KWH	BASELINE ERR	MODEL ERR
1	2026-09-28 11:00	835.6	270.3	37.6
2	2026-09-28 11:00	780.1	233.2	32.4
3	2026-09-28 11:00	783.0	228.3	28.4
4	2026-09-28 16:00	659.9	98.5	31.7
5	2026-09-28 11:00	828.2	250.9	34.0
6	2026-09-28 11:00	749.1	244.7	36.9
7	2026-09-25 11:00	751.0	14.4	27.6
8	2026-09-25 11:00	1016.6	6.0	44.9
9	2026-09-23 11:00	809.0	12.1	31.0
10	2026-09-28 11:00	743.0	240.7	31.7

The baseline wins the four peak hours whose previous day was also a weekday (R7, R8, R9 on Sep 25). It fails badly on Monday peaks (Sep 28, previous day Sunday), with errors of 230–270 kWh; the model stays within 30–40 kWh everywhere. **The GB model is the uniform choice for a live forecast.**



Screenshot 19. Forecast metrics: baseline and model MAE and RMSE, per-region MAE with the chosen configuration, and peak-hour errors.

USEFULNESS

7.5 kWh regional MAE is useful: about 1% of a 1000 kWh peak hour. The 43 kWh baseline is not useful at the hourly level. In production the main risk is delayed arrivals (2.0% of readings): predictions made at 00:00 can be revised later. A reasonable mitigation is to attach a confidence band proportional to completeness at the forecast horizon.

9 Submit the report and code

9.1 Organise the report

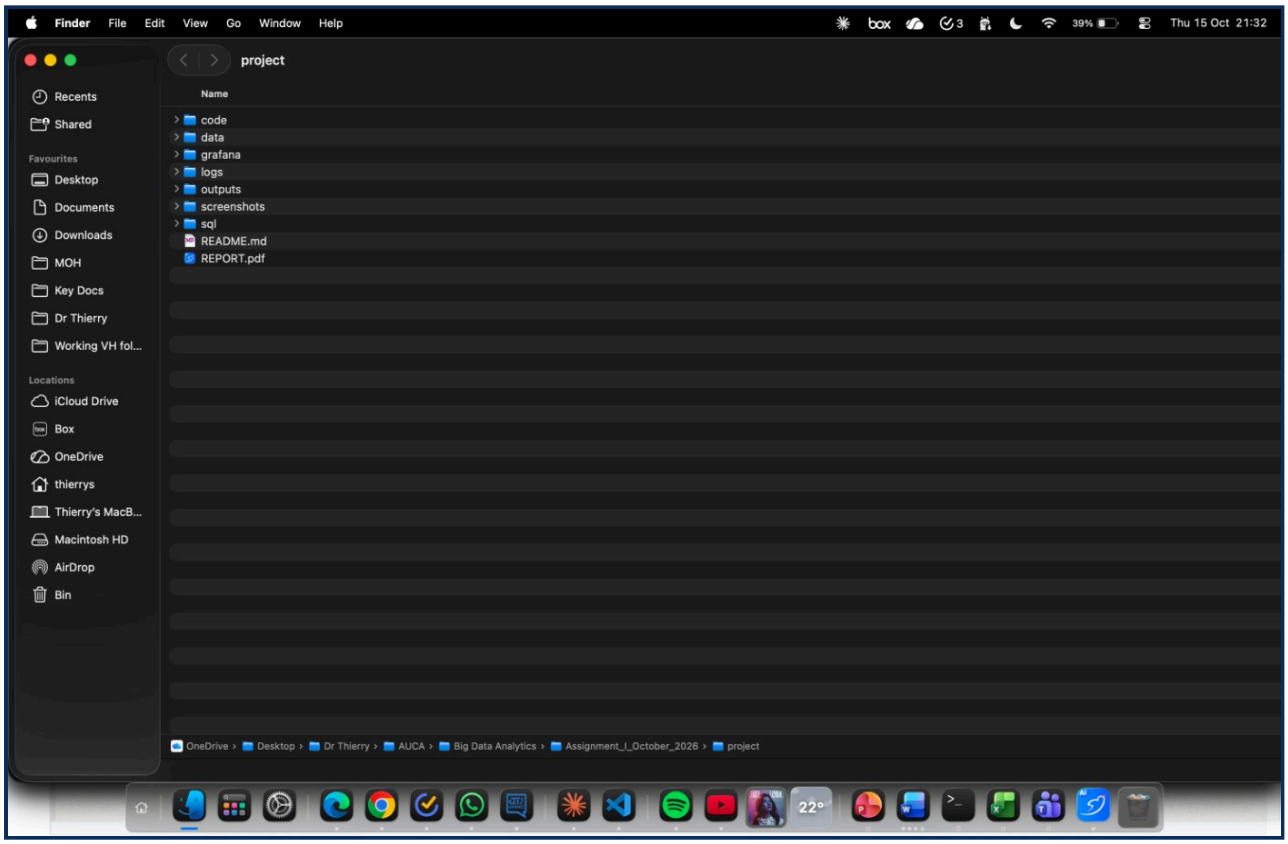
This document.

9.2 Make the work reproducible

GitHub repository: `https://github.com/code-gnosis/Group10_Big_Data_Analytics_assignment_1`

Table 27. Repository contents.

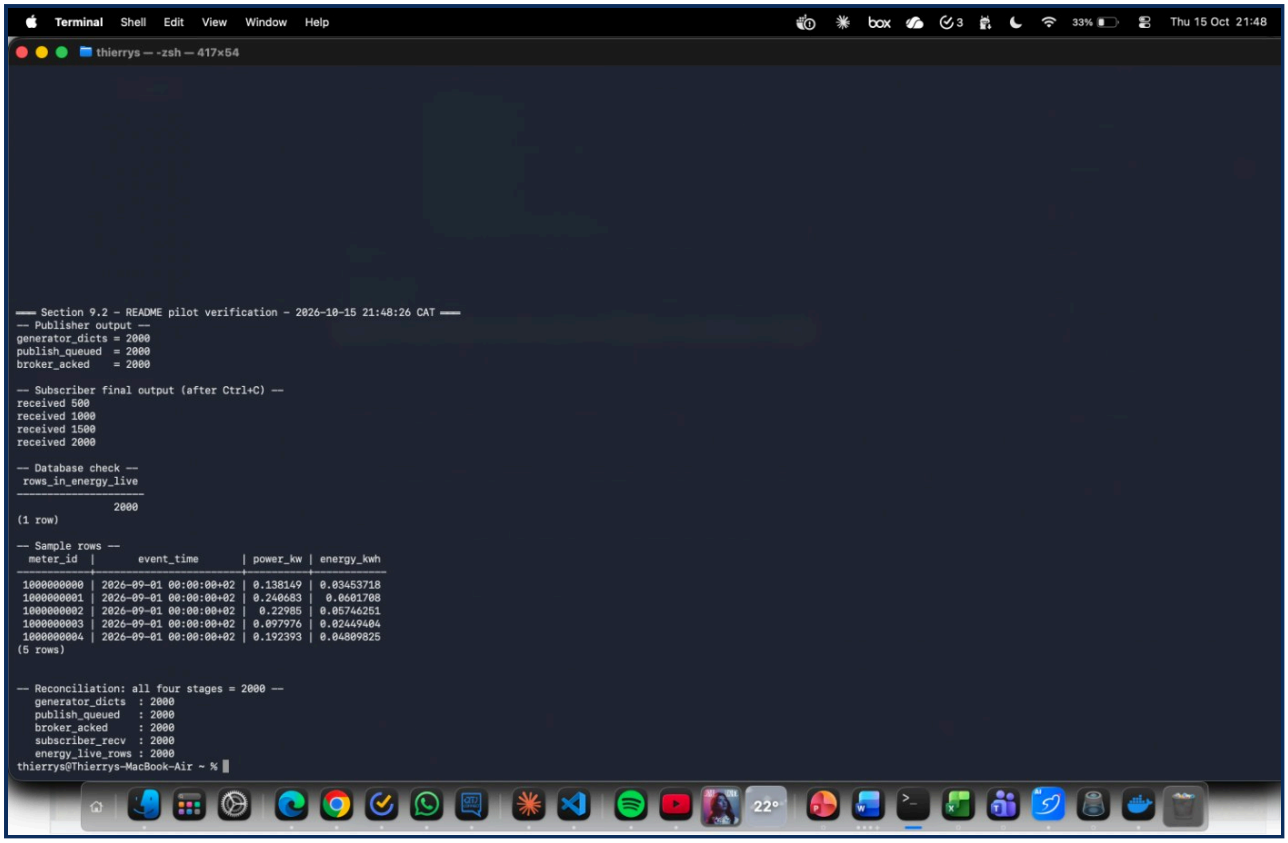
PATH	CONTENTS
`code/`	all Python scripts and env_report.sh
`code/smart_meter_simulator.py`	the common generator, unchanged
`sql/`	every PostgreSQL + TimescaleDB SQL file we ran
`grafana/`	datasource + 2 dashboards (Raw + Aggregate) provisioned as JSON
`logs/`	captured output of each step
`outputs/`	figures + JSON summaries (query timings, dashboard timings, forecast metrics)
`screenshots/`	22 screenshots embedded in this report
`README.md`	execution order and commands



Screenshot 20. The submission folder: `code`, `data`, `grafana`, `logs`, `outputs`, `screenshots` and `sql`, with `README.md` and `REPORT.pdf`.

Passwords in the SQL and Python files use the training value `gishushu`; replace it with your own secret before deploying.

README pilot verification. Section 9.2 point 4 of the brief asks for the README's pilot instructions to be run and the four counts verified.



Screenshot 21. README pilot instructions re-run, 15 Oct: publisher and subscriber output, the database check and the reconciliation, all at 2,000.

9.3 Document contributions

Table 28. Contributions.

MEMBER	STUDENT ID	CONTRIBUTIONS
Thierry SEBAKUNZI	101394	Full pipeline (sections 1, 2), chunk benchmarks (section 3), compression (section 4), report writing
Kwezi James	101502	Utility data analysis (section 5), continuous aggregates (section 6), freshness test
Nishyirimbere Yves	101388	Grafana dashboards (section 7), forecasting (section 8), result verification

Management recommendation

SECTION 5.5 · AT MOST 500 WORDS

The pilot evidence supports switching the smart-meter programme to a **broker-and-headend architecture backed by TimescaleDB**. The pilot (section 1.3) delivered 2,000 of 2,000 messages with zero loss at QoS 1, and the historical replay (section 2.1) loaded 10,696,848 rows in 80.4 seconds via CSV + COPY. Both runs fit comfortably within the daily operating budget for the full fleet.

The data-quality baseline is actionable. Of 10,752,000 planned readings, 1.01% are missing, 0.50% arrive as duplicates and 2.00% arrive late by 30–180 minutes. Missing readings are best addressed by field work rather than code. Duplicates are already handled by the section 2.3 cleaning routine, which uses `ctid` to remove exact copies in a repeatable way; the pre-check for conflicting duplicates returned zero, so the delete is safe.

For storage we recommend **1-day chunks with meter-segmented compression**. The measurements in section 4 show the storage footprint is 2.72× smaller than the uncompressed table (502 MB against 1,364 MB), and the queries that drive the dashboards get faster by 48.6% (Q2), 82.6% (Q3) and 58.8% (Q5). The single-meter query (Q4) stays within 8% of baseline because `meter_id` is a segment key. The one query that gets slower is Q1 (narrow time range), but in section 6 the continuous aggregates serve that workload, so compressed storage never has to handle it. Compression costs about 644 ms per chunk, about 13 seconds of background work per day of ingest.

For reporting we recommend **continuous aggregates for every dashboard query**. The section 7.4 measurements showed the full Raw dashboard refreshing in 8,414 ms against the Aggregate dashboard in 11.6 ms: a 725× speed-up for a one-off cost of 20 seconds of initial refresh and 18.6 MB of storage. The Raw dashboard would be unusable in a control room; the Aggregate dashboard refreshes faster than the human-perception threshold.

For the next-day load forecast, the **gradient-boosting model should be adopted**. Section 8 showed a test MAE of 7.47 kWh against 43.16 kWh for the previous-day baseline (5.8× improvement; 8.2× on RMSE). The gain is largest on the Sunday-to-Monday transition, where the baseline mis-forecasts peak hours by 230–270 kWh and the model stays within 30–40 kWh: exactly the band that matters for day-ahead balancing contracts.

THREE OPERATIONAL ACTIONS FOLLOW IMMEDIATELY

1. Deploy the headend using the configuration above, and freeze the section 2.3 cleaning as part of the ingest.
2. Stand up both Raw and Aggregate dashboards for the control room, defaulting operators to Aggregate.
3. Move the forecast model into a daily batch job that re-fits weekly and quotes a confidence band proportional to the delayed-arrival fraction at the horizon.

The combined footprint is well below the 10 GB budget cited in the brief, which leaves headroom for the next year of ingest without re-provisioning.